

Hack Assembly – Some typical operations

- D: data register
- A: address/data register
- M: currently selected memory location $M = \text{RAM}[A]$

```
// D=-1
D=-1

// D=10
@10
D=A

// D++
D=D+1

// D=RAM[17]
@17
D=M
```

```
// RAM[17]=D
@17
M=D

// RAM[17]=10
@10
D=A
@17
M=D

// RAM[5]=RAM[3]
@3
D=M
@5
M=D
```

18

Hack Assembly – Terminating a program

- An infinite loop is useful to terminate the program

```
// Program: Add2.asm
// Computes: RAM[2]=RAM[0]+RAM[1]
// Usage: put values in RAM[0], RAM[1]

0. @0
1. D=M    // D=RAM[0]

2. @1
3. D=D+M  // D=D+RAM[1]

4. @2
5. M=D    // RAM[2]=D (=RAM[0]+RAM[1])

6. @6
7. 0;JMP
```

Jump in any case: 0
is just a useless
value specified to
complete the
instruction

19

Hack Assembly – Branching

```
//Program: Signum.asm
//Computes: if R0>0
//          R1=1
//          else
//          R1=0
```

```
@R0
D=M

@POSITIVE
D;JGT

@R1
M=0
@END
0;JMP

(POSITIVE)
@R1
M=1

(END)
@END
0;JMP
```

using a label

declaring a label

NB: references to a label will be translated into references to the instruction number following that label's declaration

20

Hack Assembly – Variables

- Variables “do not exist” in low level languages
 - they are just names assigned to RAM locations

```
//Program: Flip.asm
//flips the values of
//RAM[0] and RAM[1]
//
//temp = R1
//R1 = R0
//R0 = temp
```

```
@R1
D=M
@temp
M=D //temp=R1

@R0
D=M
@R1
M=D //R1=R0

@temp
D=M
@R0
M=D //R0=temp

(END)
@END
0;JMP
```

“Please, find some available RAM location and use it to store the value of my variable temp”

NB: from now on, every occurrence of @temp will be translated into @RAM_location

21

Hack Assembly – Iteration

■ Example: compute 1+2+...+n

Pseudo-code

```
//Computes RAM[1]=1+2+...+RAM[0]
```

```
n = R0
i = 1
sum = 0
LOOP:
  if i > n goto STOP
  sum = sum + i
  i = i + 1
  goto LOOP
STOP:
  R1 = sum
```

Hack assembly code

```
// Program: Sum1toN.asm
// Computes RAM[1]=1+2+...+n
// Usage: put a number (n)
// in RAM[0]
```

```
@R0
D = M
@n
M = D // n = R0
@i
M = 1 // i = 1
@sum
M = 0 // sum = 0
...
```

```
(LOOP)
  @i
  D = M
  @n
  D = D - M
  @STOP
  D;JGT //if i>n goto STOP
```

```
@sum
D = M
@i
D = D + M
@sum
M = D //sum = sum + i
@i
M = M + 1 //i = i + 1
@LOOP
0;JMP
```

```
(STOP)
  @sum
  D = M
  @R1
  M = D // RAM[1] = sum
```

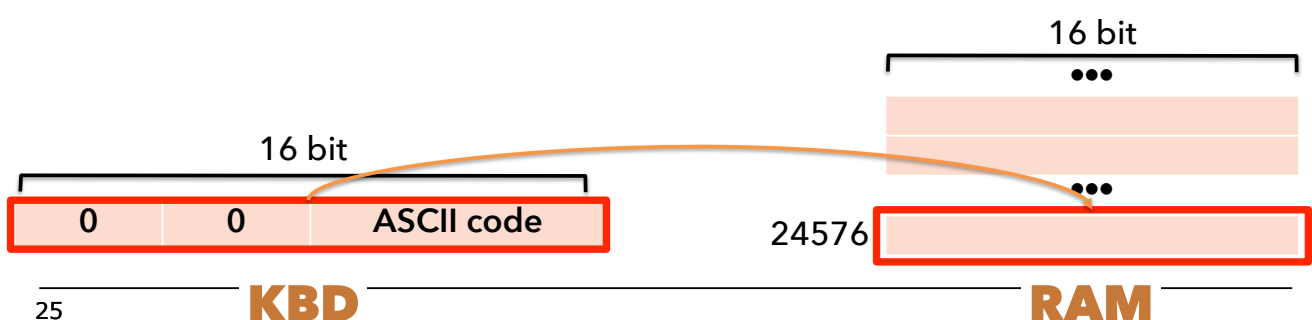
```
(END)
@END
0;JMP
```

	0	1	2	3	...
RAM[0]	3				
n:	3				
i:	1	2	3	4	...
sum:	0	1	3	6	...

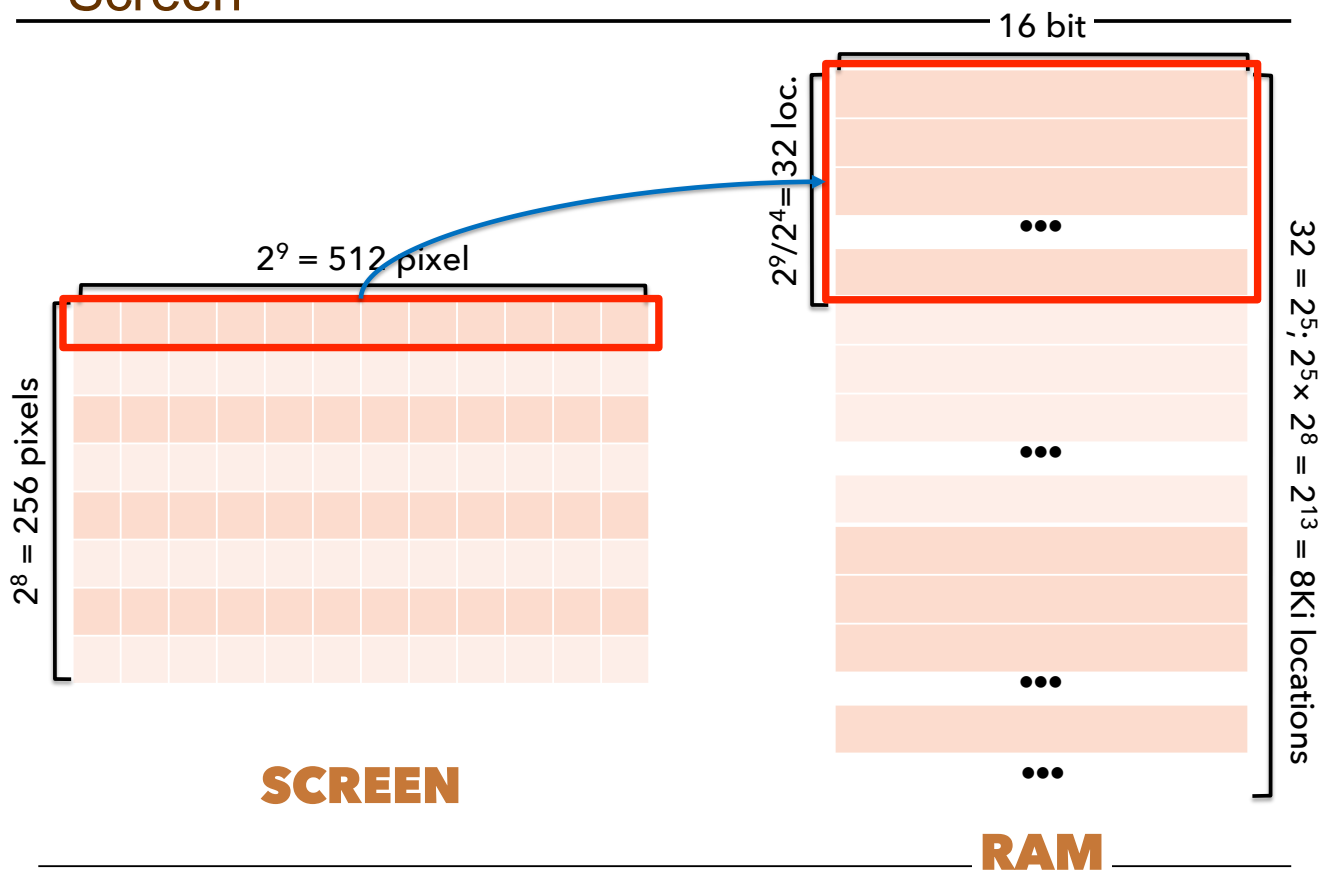
tracing

Input/Output

- Only 2 peripherals are conceived: screen and keyboard
- Input/Output is memory-mapped
 - unique address space
- Drawing pixels on the screen is writing binary values in a memory segment (including several locations)
- Listening to the keyboard is reading binary values from a memory segment (reduced to a single location)
 - ASCII code (8 bit) in a 16 bit location (RAM[24576])

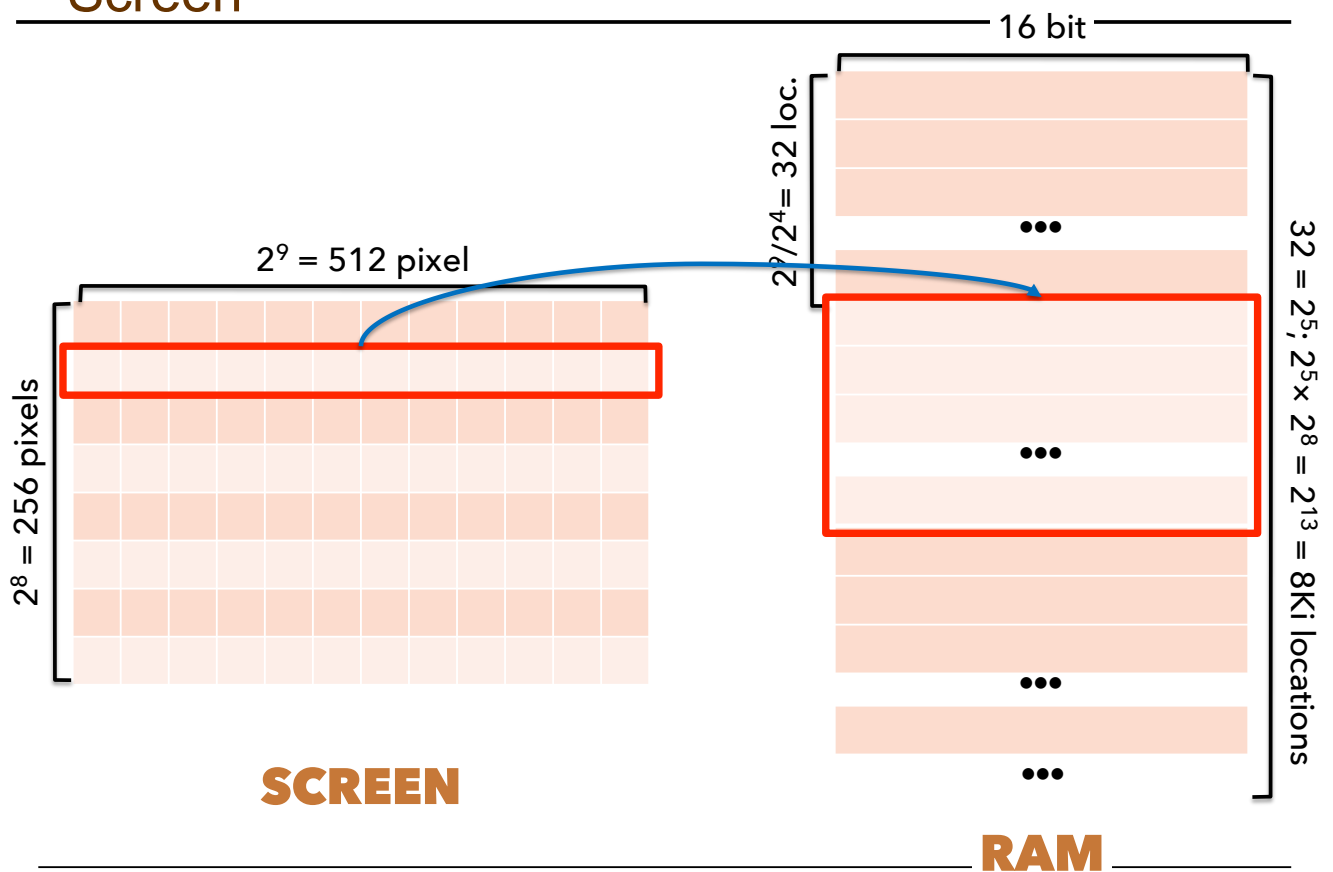


Screen



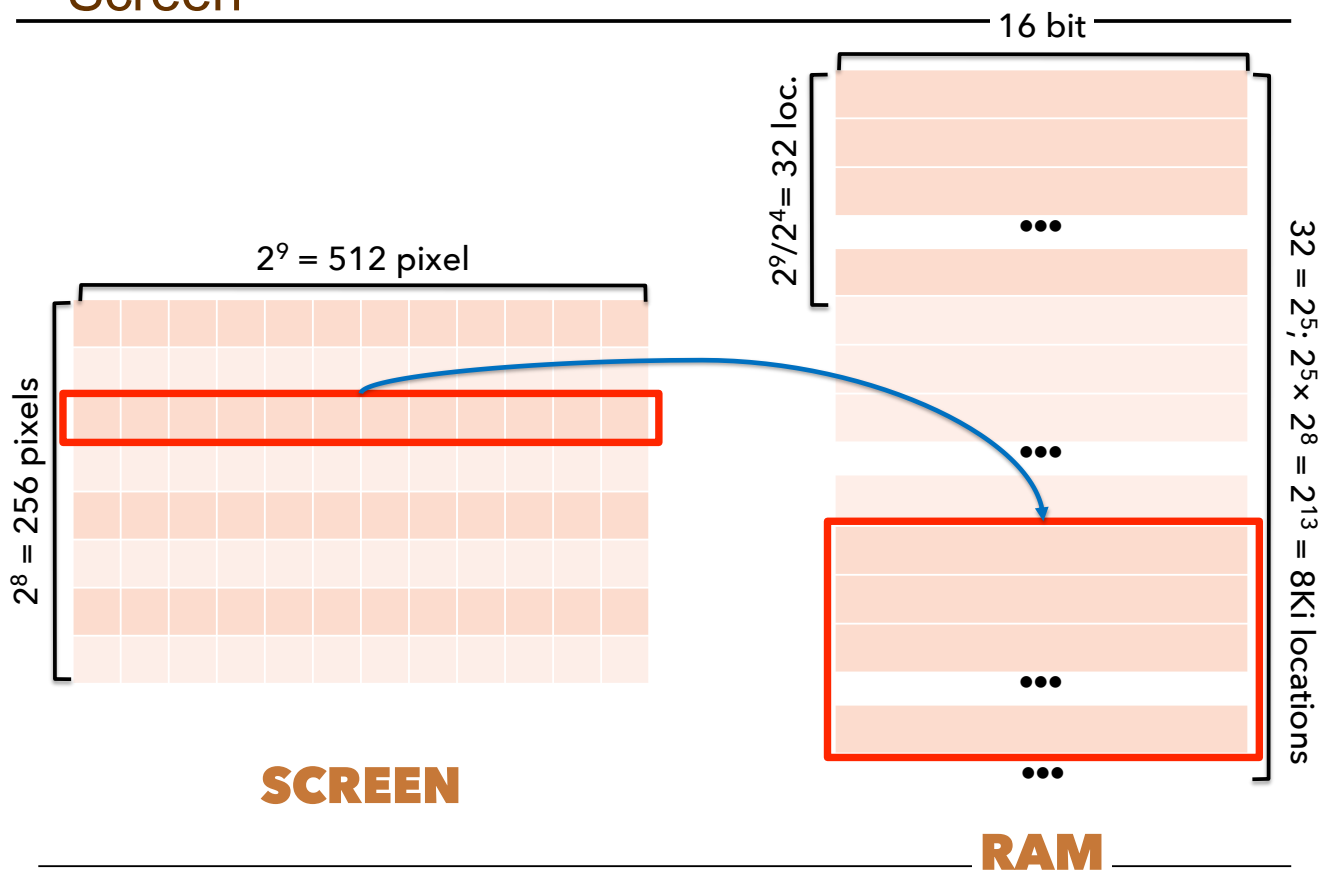
26

Screen



27

Screen



28

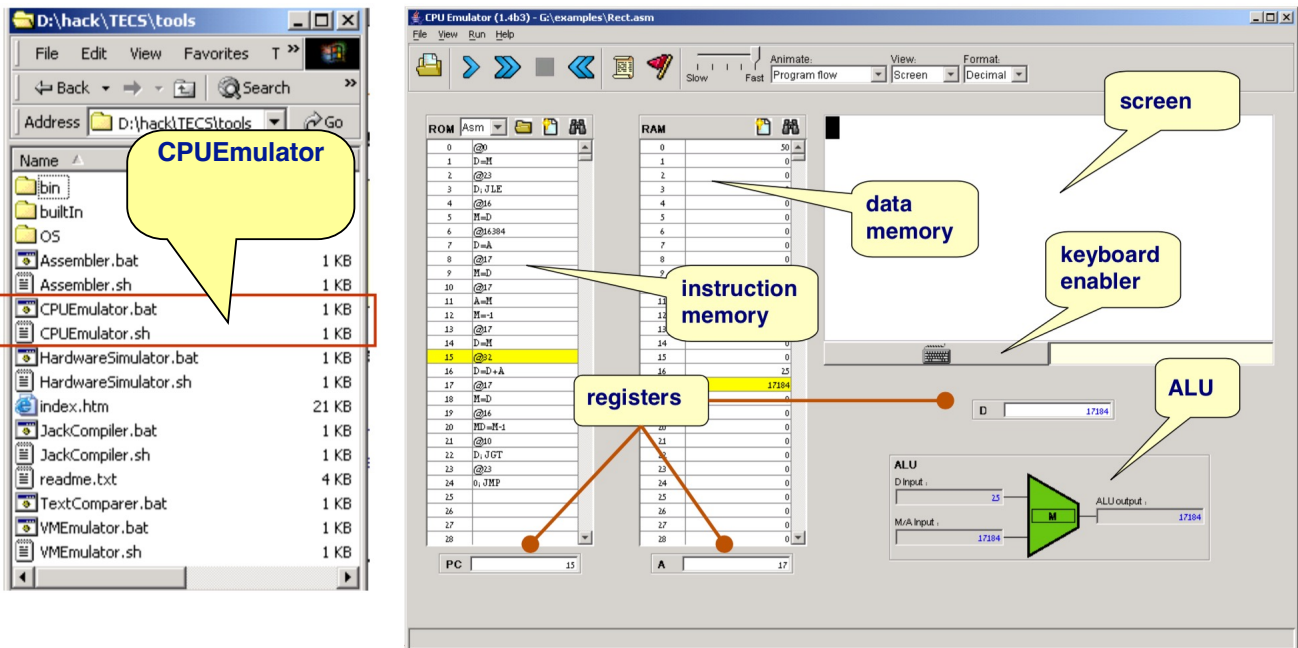
Project 4: Machine Language Programming

Program	Description	Comments / Tests
Mult.asm	Multiplication: in the Hack computer, the top 16 RAM words (RAM[0] ... RAM[15]) are also referred to as the so-called <i>virtual registers</i> R0 ... R15. With this terminology in mind, this program computes the value R0*R1 and stores the result in R2. Note that in the context of this program, we assume that R0>=0, R1>=0, and R0*R1<32768 (you are welcome to ponder where this limiting value comes from). Your program need not test these conditions, but rather assume that they hold.	Start by using the supplied assembler to translate your Mult.asm program into a Mult.hack file. To test your program, load Mult.hack into the ROM. Next, put some values in R0 and R1, run the code, and inspect R2. Alternatively, use the supplied Mult.tst script and Mult.cmp compare file (that's how we test your program). These supplied files are designed to test your program by running it on several representative data values.
Fill.asm	I/O handling: this program illustrates low-level handling of the screen and keyboard devices, as follows. The program runs an infinite loop that listens to the keyboard input. When a key is pressed (any key), the program blackens the screen, i.e. writes "black" in every pixel; the screen should remain fully black as long as the key is pressed. When no key is pressed, the program clears the screen, i.e. writes "white" in every pixel; the screen should remain fully clear as long as no key is pressed.	Start by using the supplied assembler to translate your Fill.asm program into a Fill.hack file. Implementation note: your program may blacken and clear the screen's pixels in any spatial/visual order, as long as pressing a key continuously for long enough results in a fully blackened screen, and not pressing any key for long enough results in a fully cleared screen. The simple Fill.tst script, which comes with no compare file, is designed to do two things: (i) load the Fill.hack program, and (ii) remind you to select 'no animation', and then test the program interactively by pressing and releasing some keyboard keys.

29

Project 4: Testing notes

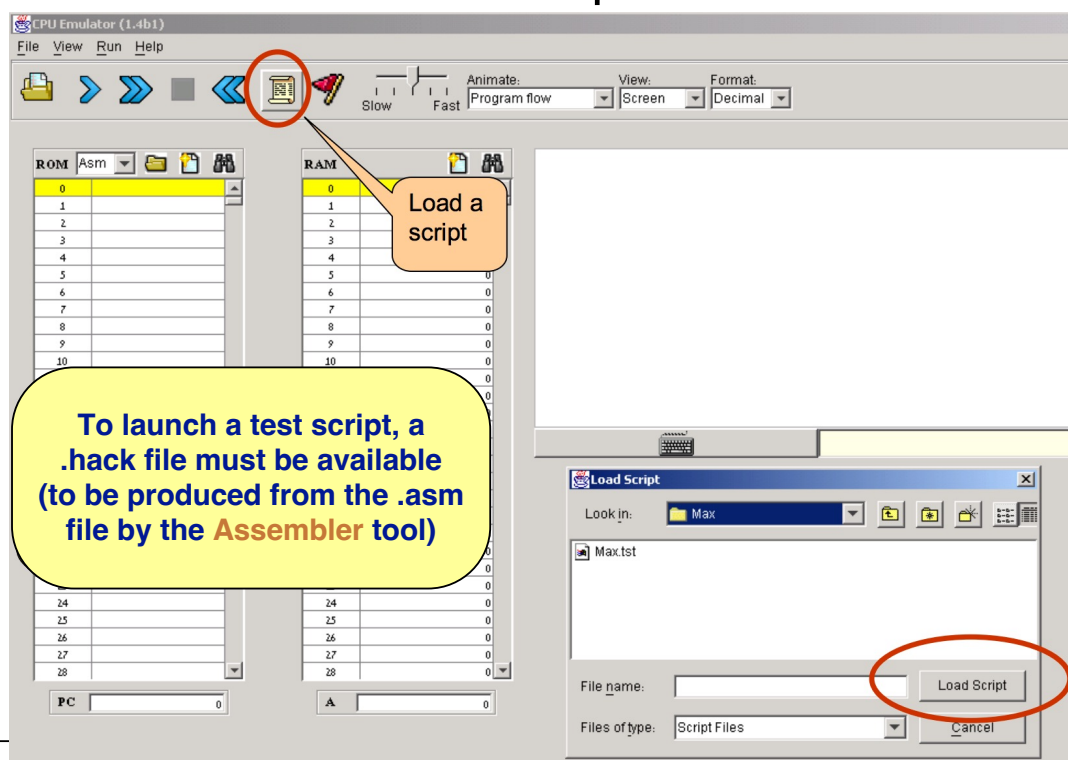
- A different tool must be used: **CPUEmulator**



30

Project 4: Testing notes

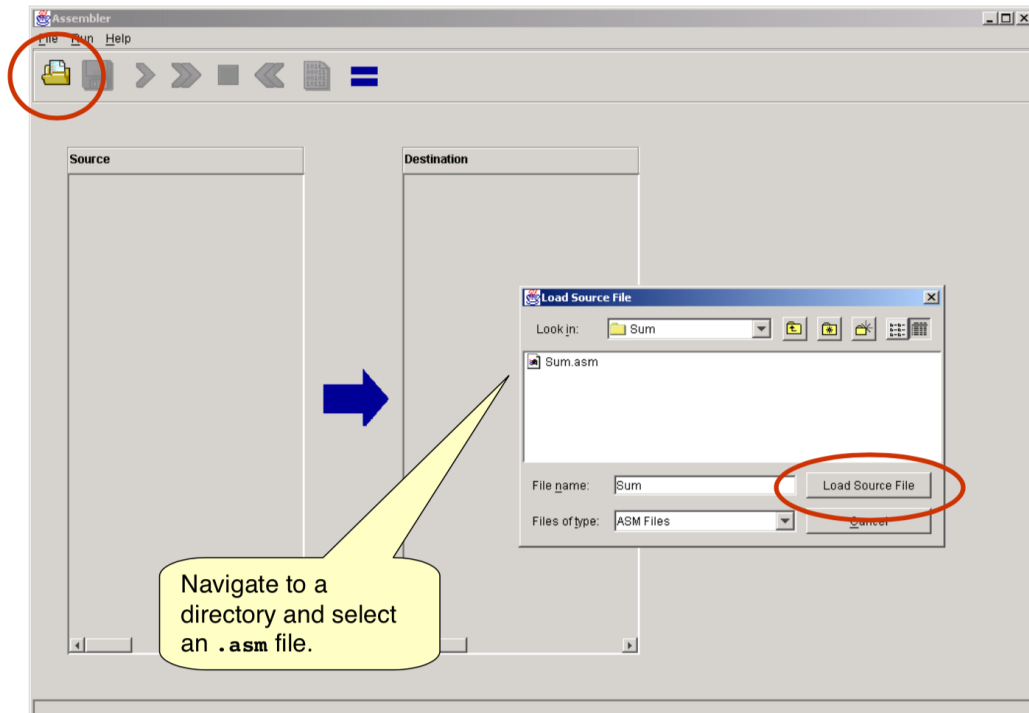
- CPUEmulator and test scripts



31

Project 4: Testing notes

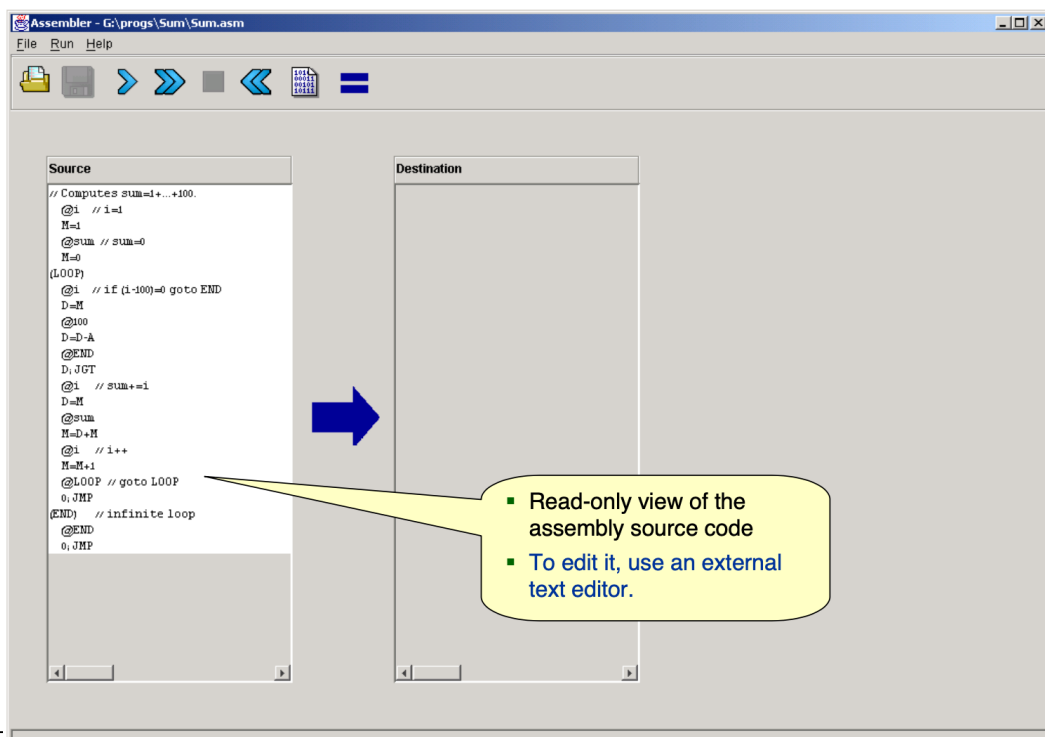
■ Another new tool: the Assembler tool



32

Project 4: Testing notes

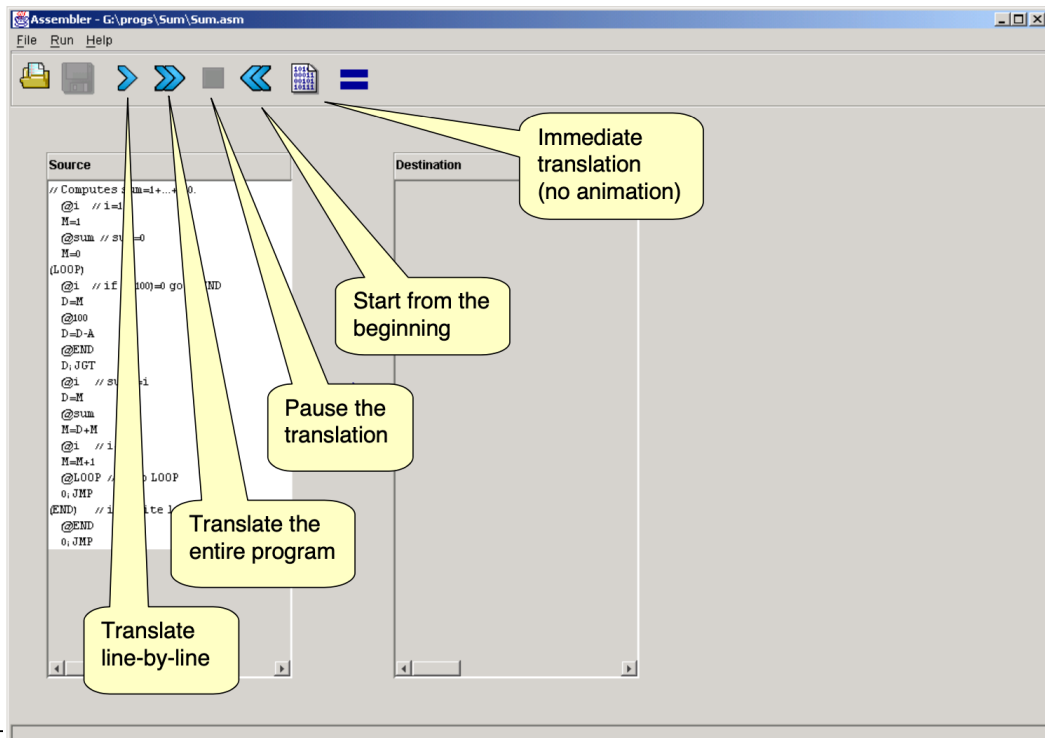
■ The Assembler tool



33

Project 4: Testing notes

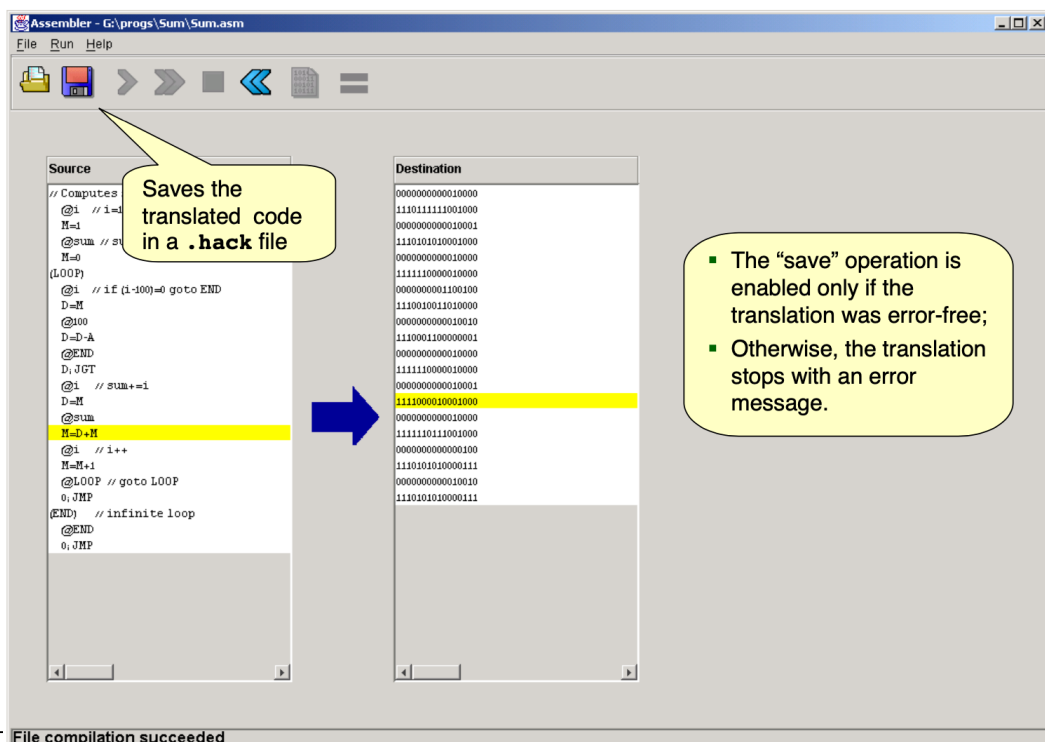
■ The Assembler tool



34

Project 4: Testing notes

■ The Assembler tool



35

With a little help from my friends - Mult.asm

- The ALU of the Hack computer does not include multiplication functions
 - Mult.asm is a way to extend the computer capabilities on the **software** side
- Multiplication comes from a sequence of sums
 - Accumulate the results of several additions by performing a **loop**
 - Remember to keep (or move in the end) the final result in **R2**
 - The test file initializes $R2 = -1$
 - $R2 = 0$ cannot be assumed

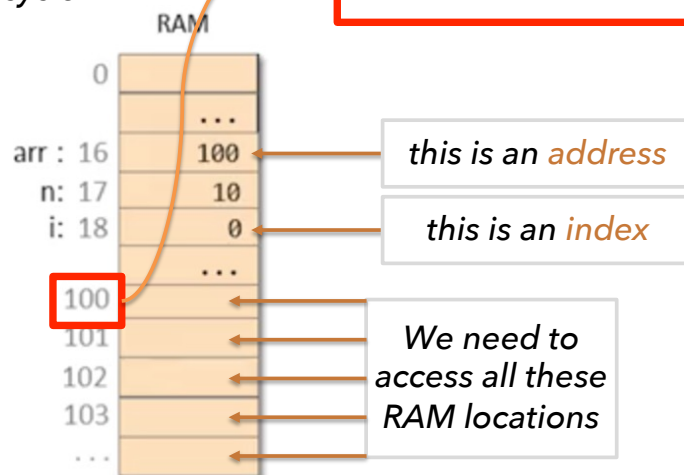
36

With a little help from my friends – Pointers

```
// for (i=0; i<n; i++){  
//     arr[i] = -1  
// }  
//  
// Suppose arr=100, n=10
```

Preparing the for cycle

```
//arr=100  
@100  
D=A  
@arr  
M=D  
  
//n=10  
@10  
D=A  
@n  
M=D  
  
//i=0  
@i  
M=0
```



37

With a little help from my friends – Pointers

```
// for (i=0; i<n; i++){
//     arr[i] = -1
// }
//
// Suppose arr=100, n=10
```

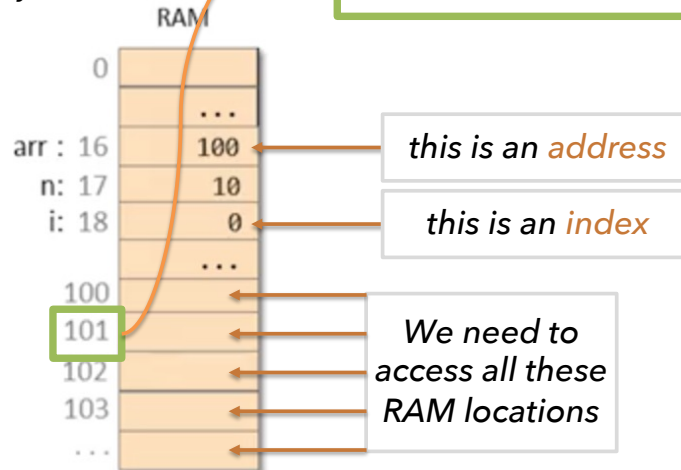
obtained by:
`arr + i` `(100+1)`
 (at second iteration, $i=1$)

Preparing the for cycle

```
//arr=100
@100
D=A
@arr
M=D

//n=10
@10
D=A
@n
M=D

//i=0
@i
M=0
```



38

With a little help from my friends – Pointers

```
// for (i=0; i<n; i++){
//     arr[i] = -1
// }
//
// Suppose arr=100, n=10
```

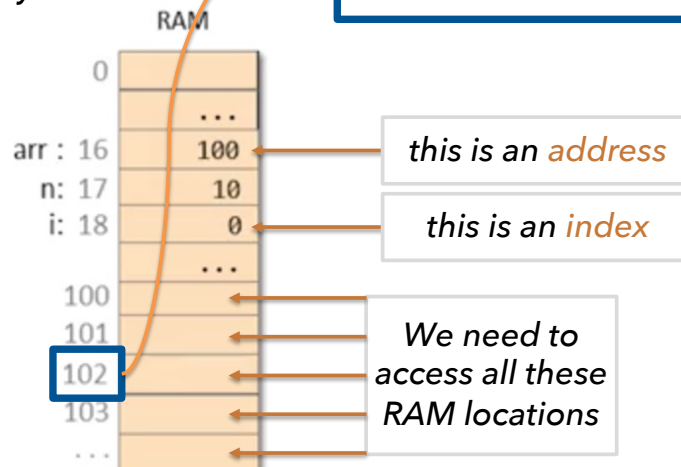
obtained by:
`arr + i` `(100+2)`
(at third iteration, i=2)

Preparing the for cycle

```
//arr=100
@100
D=A
@arr
M=D

//n=10
@10
D=A
@n
M=D

//i=0
@i
M=0
```



39

With a little help from my friends – Pointers

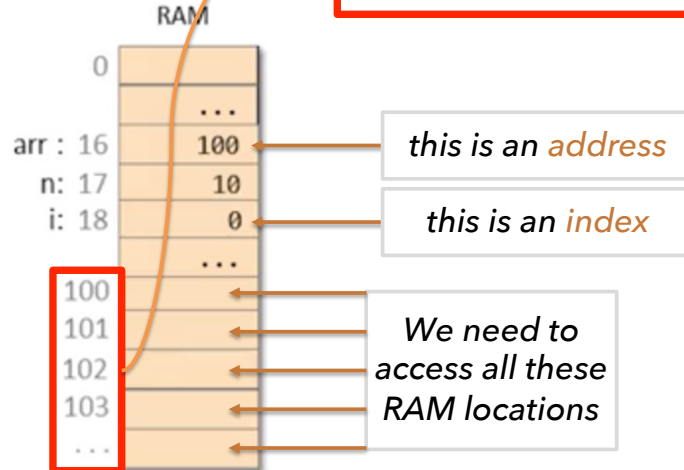
```
// for (i=0; i<n; i++){
//     arr[i] = -1
// }
//
// Suppose arr=100, n=10
```

Preparing the for cycle

```
//arr=100
@100
D=A
@arr
M=D

//n=10
@10
D=A
@n
M=D

//i=0
@i
M=0
```



40

With a little help from my friends – Pointers

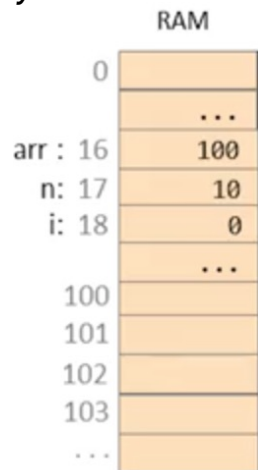
```
// for (i=0; i<n; i++){
//     arr[i] = -1
// }
//
// Suppose arr=100, n=10
```

Preparing the for cycle

```
//arr=100
@100
D=A
@arr
M=D

//n=10
@10
D=A
@n
M=D

//i=0
@i
M=0
```



```
(LOOP)
//if i==n goto END
@i
D=M
@n
D=D-M
@END
D;JEQ
```

```
//RAM[arr+i]=-1
@arr
D=M
@i
A=D+M
M=-1
```

```
//i++
@i
M=M+1

@LOOP
0;JMP
```

```
(END)
@END
0;JMP
```

this is [arr+i]
(an address)
and goes to A

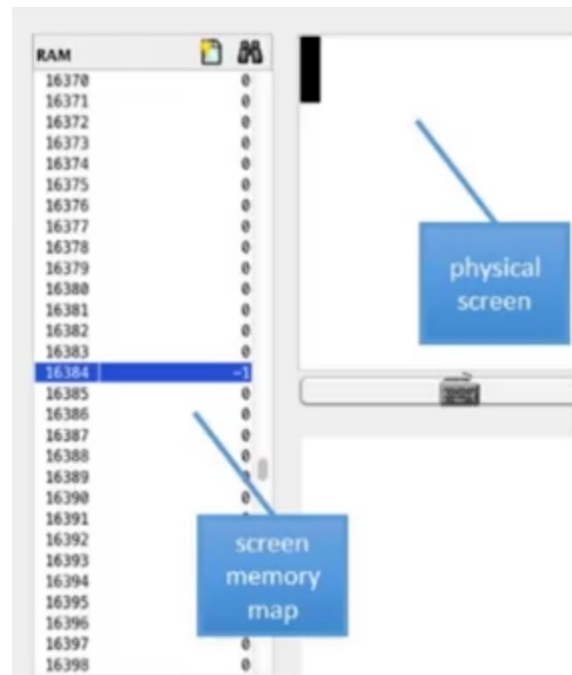
42

With a little help from my friends – Managing I/O

■ Example: Program Rectangle.asm

■ Pseudo-code:

```
// for (i=0; i<n; i++) {  
//     draw 16 black pixels at the  
//     beginning of row i  
// }  
  
addr = SCREEN  
n = RAM[0]  
i = 0  
  
LOOP:  
    if i > n goto END  
    RAM[addr] = -1 // 1111111111111111  
    // advances to the next row  
    addr = addr + 32  
    i = i + 1  
    goto LOOP  
  
END:  
    goto END
```



43

With a little help from my friends – Managing I/O

■ Run Rectangle.asm in the CPUEmulator

■ Pay attention to the structure of the program and to its working process in action

44

With a little help from my friends – Delivery instructions

- Please adhere to the following notation for delivering the .asm files
 - preserve the directory structure while working on your project files
 - Mult.asm and related files are inside the “mult” folder
 - Fill.asm and related files are inside the “fill” folder
 - submit one zip file
 - _P04CognomeNome.zip (containing both Mult.asm and Fill.asm)
 - The zip file must contain immediately the asm. files only, with no additional file or directory

Perspective

- Hack is a simple machine language
- User friendly syntax: $D=D+A$ instead of `ADD D,D,A`
- Hack is a “ $\frac{1}{2}$ -address machine”: any operation that needs to operate on the RAM must be specified using two commands: an **A**-command to address the RAM, and a subsequent **C**-command to operate on it
- A Macro-language can be easily developed
- A Hack assembler is needed and will be discussed and developed later in the course.